

B.Sc. ECS – I Sem – II
Introduction to Microcontrollers and Embedded System
Electronics Practical

1 Create a simple project in Keil MicroVision. Write, compile, and debug a basic program to control an LED. Simulate the program and analyze the results.

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

To control an LED using the 8051 microcontroller, we can write an assembly language program (ALP) to turn the LED on and off. The LED will be connected to one of the GPIO pins, such as Port 1 (P1.0), and we will control this pin through the assembly code.

Here's an ALP to turn the LED on and off:

assembly

Copy

ORG 0H ; Start address for the program

START:

MOV P1, #00H ; Initialize Port 1 to low (LED OFF)

MOV R0, #00H ; Counter or delay variable

DELAY:

NOP ; No operation for delay

NOP ; No operation for delay

NOP ; No operation for delay

NOP ; No operation for delay

INC R0 ; Increment the counter

MOV A, R0 ; Move R0 to accumulator

CJNE A, #50H, DELAY ; Check if delay counter reaches 50H (80 in decimal)

MOV P1, #FFH ; Set Port 1 to high (LED ON)

MOV R0, #00H ; Reset the counter

DELAY2:

NOP ; No operation for delay

NOP ; No operation for delay

NOP ; No operation for delay

NOP ; No operation for delay

INC R0 ; Increment the counter

MOV A, R0 ; Move R0 to accumulator

CJNE A, #50H, DELAY2 ; Check if delay counter reaches 50H

MOV P1, #00H ; Set Port 1 to low (LED OFF)

SJMP START ; Repeat the process from the start (infinite loop)

END

Explanation of the program:

1. **ORG 0H:** This specifies the starting address of the program, which is address 0x00 in memory.
2. **MOV P1, #00H:** Initializes Port 1 (P1) to 00H, turning the LED off (since the LED is typically connected to a low pin).
3. **NOP:** These are No Operation instructions, used to create a small delay. In practice, you would use more precise delay techniques such as timers, but this simple method works for small delays.
4. **MOV R0, #00H:** Initializes register R0 to 0, which is used as a counter.

5. **DELAY:** A loop that runs and waits until a certain delay time has passed (controlled by R0 reaching a value of 50H).
6. **CJNE A, #50H, DELAY:** Compares the value in the accumulator with 50H (80 in decimal) and if they are not equal, it jumps back to DELAY to keep the loop running.
7. **MOV P1, #FFH:** Sets Port 1 to FFH, which turns the LED on.
8. **SJMP START:** This instruction causes an infinite loop by jumping back to the start of the program to repeat the process.

How the LED is controlled:

- Initially, the LED is off (Port 1 is set to low 00H).
- After a delay, Port 1 is set to high (FFH), which turns the LED on.
- Another delay occurs, and then the LED is turned off again.
- This cycle repeats indefinitely.

You can adjust the delay by changing the number 50H to a higher or lower value or using more precise delay methods like timers for more accuracy.

2. Download and install Flash Magic from the official website. Launch the Flash Magic application. Select the target microcontroller. Configure the communication settings (COM port, baud rate, etc.). Load the compiled HEX file generated from Keil Micro Vision.

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

To download and install **Flash Magic** and use it for programming an 8051 microcontroller, follow the steps below:

Step-by-Step Procedure to Download, Install, and Configure Flash Magic:

1. Download Flash Magic:

1. Open your web browser and go to the official Flash Magic website:
 - **URL:** <https://www.flashmagictool.com/>
 2. On the homepage, find and click the **Download** button to download the latest version of Flash Magic.
 3. Once the download is complete, locate the installer file on your computer (usually in the **Downloads** folder).
 4. Double-click on the installer file to begin the installation process.
-

2. Install Flash Magic:

1. **Run the Installer:** Double-click on the downloaded **Flash Magic Setup** file.
 2. **Follow the Installation Instructions:**
 - You may need to accept the terms and conditions.
 - Choose the installation directory where you want to install Flash Magic, or use the default location.
 3. Click **Install** to start the installation. Wait for it to complete.
 4. Once installed, click **Finish** to exit the installer.
-

3. Launch Flash Magic:

1. After installation, you can launch Flash Magic either by:
 - Finding it in your **Start Menu** or **Desktop shortcut** (if created during installation).
 - Alternatively, go to **Start > All Programs** and look for **Flash Magic**.
-

4. Select the Target Microcontroller:

1. **Launch Flash Magic.**
 2. **Choose the Target Device:**
 - In the main window of Flash Magic, locate the "**Device**" dropdown menu.
 - Choose your specific 8051 microcontroller (for example, **AT89C51**, **AT89S52**, etc.) from the list of available microcontrollers.
-

5. Configure Communication Settings:

To set up the communication between your PC and the 8051 microcontroller:

1. **Select the COM Port:**
 - Choose the appropriate **COM port** where your 8051 programmer (or USB-to-Serial adapter) is connected.
 - If you are unsure about the COM port, you can check it in **Device Manager** under **Ports (COM & LPT)**.
2. **Set the Baud Rate:**
 - Set the **Baud Rate** according to the communication settings for your 8051 microcontroller. Common baud rates are **9600**, **19200**, or **115200**. Refer to your 8051 microcontroller datasheet for the exact baud rate it supports.
3. **Select the Protocol:**
 - Choose the appropriate **programming protocol** (e.g., **ISP**, **UART**, etc.) that matches your hardware setup. Flash Magic typically supports the serial interface (UART) for the 8051.
4. **Set Other Options:**
 - If necessary, select additional options like "**Erase All Flash**" or "**Verify after Programming**" depending on your needs.

6. Connect the Programmer (Optional):

If you're using an external programmer or USB-to-Serial adapter, ensure it's connected to the microcontroller and the PC.

- Ensure that the **8051 microcontroller is in ISP mode** (In-System Programming), usually done by setting the **RST pin** low during reset or other specific procedures (refer to the microcontroller datasheet for this).

7. Start Programming:

Once the target device and communication settings are configured, you can:

1. **Load the Firmware File:**
 - Click on **File > Open** to load the hex file or program you want to burn into the microcontroller.
2. **Start Programming:**
 - Click on **Start** or **Begin** to initiate the programming process. Flash Magic will upload the firmware to the microcontroller via the selected COM port.
3. Wait for Flash Magic to complete the programming process. After successful programming, you should see a confirmation message.

8. Verify and Disconnect:

1. Once the programming is complete, you can verify the program by selecting the **Verify** option (optional).
2. After programming, disconnect the 8051 microcontroller or disconnect the programming tool from the COM port.

Conclusion:

That's it! You've successfully downloaded, installed, configured, and used Flash Magic to program an 8051 microcontroller. Always ensure that your communication settings match those of your specific hardware to avoid connection issues.

3. Write a program to demonstrate the use of internal memory, registers, and Special Function Registers (SFRs) in the 8051 microcontrollers.

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

```
ORG 0000h
mov a,#05
mov b,#03
```

```
add a,b
mov 10h,a
mov r1,10h
end
```

```
ORG 0000h
mov a,#05
mov b,#03
add a,b
mov 50h,a
mov r4,50h
end
```

Result :

1. Accumulator =
2. Memory location 10h =
3. Register r1 =

1. Accumulator =
2. Memory location 50h =
3. Register r4 =

4. Write and execute ALP to add, subtract, two 8 bit numbers.

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

Addition : Add given numbers and store result at memory 10h and register r1

```
ORG 0000h
mov a,#05
mov b,#03
add a,b
mov 10h,a
mov r1,10h
end
```

Subtraction : Subtract given numbers and store result at memory 40h and Port 3

```
mov a,#05h
mov b,#02h
subb a,b
mov 40h,a
mov p3,a
end
```

Result :

- Addition –
1. Accumulator =
 2. Memory location 10h =
 3. Register r1 =

- Subtraction –
1. Accumulator =
 2. Memory location 40h =
 3. Port p3 =

5. Write and execute ALP to multiply, and divide two 8 bit numbers.

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

Multiplication : Multiply given numbers and store result at memory P2 and register r3

```
ORG 0000h
mov a,#03
mov b,#03
mul a,b
mov p2,a
mov r3,p2
end
```

Division: Add given numbers and store result at memory 30h and Port 0

```
mov a,#08h
mov b,#02h
div a,b
mov 30h,a
mov p0,a
end
```

Result :

Multiplication – 1. Accumulator =
2. Port p0 =
3. Register r3 =

Subtraction –1. Accumulator =
2. Memory location 30h =
3. Port p0 =

6. Write and execute ALP to implement logical operations like AND, OR, XOR on data.

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

AND : Multiply given numbers and store result at memory Port 2 and register r3

```
ORG 0000h
mov a,#03h
mov b,#03h
anl a,b
mov p2,a
mov r3,p2
end
```

OR: Add given numbers and store result at memory 30h and Port 0

```
mov a,#08h
mov b,#02h
orl a,b
```

```

mov 20h,a
mov p1,a
end

```

XOR: Add given numbers and store result at memory 30h and Port 0

```

mov a,#08h
mov b,#02h
xrl a,b
mov 50h,a
mov p0,a
end

```

Result :

- | | |
|-------|--------------------------|
| AND – | 1. Accumulator = |
| | 2. Port p2 = |
| | 3. Register r3 = |
| OR – | 1. Accumulator = |
| | 2. Memory location 20h = |
| | 3. Port p1 = |
| XOR – | 1. Accumulator = |
| | 2. Memory location 50h = |
| | 3. Port p0 = |

7. Write a program to configure and use the ports of the 8051 microcontrollers.

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

Port 0 :

```

ORG 0000h
mov a,#03
mov b,#03
mov p0,a
mov r3,p0
end

```

Port 1 :

```

ORG 0000h
mov a,#03
mov b,#03
mov p1,a
mov r3,p1
end

```

Port 2 :

```

ORG 0000h
mov a,#03
mov b,#03
mov p2,a
mov r3,p2
end

```

Port 3 :

```

ORG 0000h
mov a,#03
mov b,#03

```

```

mov p3,a
mov r3,p3
end

```

Result :

- | | |
|----------|------------------|
| Port 0– | 1. Accumulator = |
| | 2. Port p0 = |
| | 3. Register r3 = |
| Port 1– | 1. Accumulator = |
| | 2. Port 1 = |
| | 3. Register r3 = |
| Port 2 – | 1. Accumulator = |
| | 2. Port 2 = |
| | 3. Register r3 = |
| Port 3 – | 1. Accumulator = |
| | 2. Port 3 = |
| | 3. Register r3 = |

8. Write a program to configure and use the timer/counter functionality of the 8051 microcontrollers.

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

To configure and use the **timer/counter** functionality of the **8051 microcontroller**, you can use the timers (Timer 0 and Timer 1) for various purposes like generating delays, counting events, etc. Here's an example program that uses **Timer 0** in **mode 1** (16-bit timer mode) to generate a delay, which is a common use case:

8051 Timer/Counter Program Example

This program will configure **Timer 0** in **mode 1** (16-bit mode), generate a delay, and toggle an LED connected to **P1.0**.

Code:

assembly

Copy

ORG 0H ; Starting address of the program

START:

```

MOV TMOD, #01H ; Configure Timer 0 in Mode 1 (16-bit mode)
MOV TH0, #0xFC ; Load high byte of Timer 0 with 0xFC (for delay)
MOV TL0, #0x66 ; Load low byte of Timer 0 with 0x66 (for delay)
SETB TR0 ; Start Timer 0
MOV P1, #00H ; Initialize Port 1 (LED OFF)

```

WAIT:

```

JNB TF0, WAIT ; Wait until Timer 0 overflow (TF0 flag set)
CLR TF0 ; Clear Timer 0 overflow flag
CPL P1.0 ; Toggle P1.0 (LED)
MOV TH0, #0xFC ; Reload Timer 0 with the same value to repeat delay
MOV TL0, #0x66
SJMP WAIT ; Infinite loop to keep toggling the LED

```

END

Explanation:

1. TMOD Register:

- MOV TMOD, #01H: This sets **Timer 0** to **Mode 1** (16-bit mode), which allows the timer to count from 0 to 65535.

2. TH0 and TL0 Registers:

- MOV TH0, #0xFC and MOV TL0, #0x66: These load the **high** and **low** bytes of the timer registers with the values 0xFC and 0x66, respectively. This makes the timer count from 0xFCC6 (or 64518) down to 0. In **Mode 1**, the timer counts from 0xFFFF to 0x0000.

3. **TR0 Register:**
 - SETB TR0: This starts **Timer 0** by setting the **TR0** bit in the **TCON** register.
4. **TF0 Flag:**
 - JNB TF0, WAIT: This instruction checks the **TF0** flag (Timer 0 overflow flag). It will **jump** back to the WAIT loop until the flag is set, meaning the timer has overflowed.
5. **TF0 Flag Clearing:**
 - CLR TF0: Once the timer overflows, we clear the **TF0** flag to reset it for the next cycle.
6. **CPL P1.0:**
 - CPL P1.0: This toggles the **P1.0** pin. If connected to an LED, this will turn the LED on and off.
7. **Timer Reload:**
 - MOV TH0, #0xFC and MOV TL0, #0x66: Reloads the timer with the initial values to repeat the delay.
8. **Infinite Loop:**
 - SJMP WAIT: This creates an infinite loop to keep the program running and toggling the LED continuously.

How it Works:

- Timer 0 is configured to overflow after a certain period, determined by the values loaded into **TH0** and **TL0**.
- Once Timer 0 overflows, the overflow flag **TF0** is set, which triggers the code to toggle the LED on **P1.0**.
- The program reloads the timer and continues this process, generating a periodic toggle of the LED.

How to Change the Delay:

- To change the delay, you can modify the values of **TH0** and **TL0**. These values determine how long the timer takes to overflow.
- The timer counts from 0xFFFF down to 0x0000, so changing the values will modify how fast the overflow happens. For example, setting **TH0** to 0xFF and **TL0** to 0x00 will give a shorter delay, while setting them to 0xFD and 0x00 will result in a longer delay.

This is a simple example of using the timer to generate a delay. In more advanced applications, timers can be used for counting external events (using the counter mode), generating time-based interrupts, or creating precise time delays for communication protocols.

9. Write and test simple programs to use timers for delay generation.

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

Here's how you can write and test simple programs using timers for delay generation in the **8051 microcontroller**. We'll use **Timer 0** in **Mode 1** (16-bit mode) for the delay generation. The delay will toggle an LED connected to **P1.0**.

Steps to Program Timers for Delay Generation:

We will write the code in **assembly** and test it by toggling an LED connected to **P1.0**.

Program 1: Basic Delay Using Timer 0 (16-bit Mode)

This program uses **Timer 0** in **Mode 1** (16-bit mode) to create a delay. It will toggle the LED on **P1.0** every time the timer overflows.

Code:

assembly

Copy

ORG 0H ; Starting address of the program

START:

```
MOV TMOD, #01H ; Configure Timer 0 in Mode 1 (16-bit timer mode)
MOV TH0, #0xFC ; Load Timer 0 high byte (for delay)
MOV TL0, #0x66 ; Load Timer 0 low byte (for delay)
SETB TR0 ; Start Timer 0
```

```

MOV P1, #00H    ; Initialize Port 1 (LED OFF)
WAIT:
JNB TF0, WAIT   ; Wait until Timer 0 overflow (TF0 flag set)
CLR TF0         ; Clear Timer 0 overflow flag
CPL P1.0        ; Toggle P1.0 (LED)
MOV TH0, #0xFC  ; Reload Timer 0 high byte (for delay)
MOV TL0, #0x66  ; Reload Timer 0 low byte (for delay)
SJMP WAIT       ; Infinite loop to keep toggling the LED
END

```

Explanation:

1. **TMOD Register:** Configures Timer 0 in **Mode 1** (16-bit timer mode). This means the timer counts from 0xFFFF to 0x0000 and overflows.
2. **TH0 and TL0 Registers:** Set the initial value for the timer. In this case, 0xFC and 0x66 will create a delay. The timer will count down from 0xFCC6 (64518) to 0, causing an overflow after a specific time.
3. **TR0 (Timer Run):** SETB TR0 starts Timer 0.
4. **TF0 (Timer Overflow Flag):** The program waits in a loop (WAIT) until **TF0** is set, indicating that the timer has overflowed.
5. **CPL P1.0:** Toggles **P1.0** (connected to an LED), turning it on or off.
6. **Reloading the Timer:** After the overflow, the timer is reloaded with the same values (0xFC and 0x66) to keep the delay going.
7. **SJMP WAIT:** The program jumps back to the WAIT loop to keep toggling the LED.

Program 2: Timer 0 with Longer Delay

You can modify the delay by adjusting the values loaded into **TH0** and **TL0**. For example, using 0xFA and 0x00 will create a longer delay compared to the previous program.

Code:

assembly

Copy

```

ORG 0H          ; Starting address of the program

```

START:

```

MOV TMOD, #01H  ; Configure Timer 0 in Mode 1 (16-bit timer mode)
MOV TH0, #0xFA  ; Load Timer 0 high byte (for a longer delay)
MOV TL0, #0x00  ; Load Timer 0 low byte (for a longer delay)
SETB TR0        ; Start Timer 0
MOV P1, #00H    ; Initialize Port 1 (LED OFF)

```

WAIT:

```

JNB TF0, WAIT   ; Wait until Timer 0 overflow (TF0 flag set)
CLR TF0         ; Clear Timer 0 overflow flag
CPL P1.0        ; Toggle P1.0 (LED)
MOV TH0, #0xFA  ; Reload Timer 0 high byte (for a longer delay)
MOV TL0, #0x00  ; Reload Timer 0 low byte (for a longer delay)
SJMP WAIT       ; Infinite loop to keep toggling the LED

```

END

How the Delay Works:

1. **TH0 = 0xFA and TL0 = 0x00:** This will make the timer start counting from 0xFA00 (64000) down to 0x0000. This will give a **longer delay** than the previous program, which used 0xFCC6 (64518).
2. The rest of the program functions the same, toggling **P1.0** (LED) after the timer overflows.

Testing the Program:

1. **Load the Program into the 8051:**
 - Use a programmer like **Flash Magic** or **Keil uVision** to compile and load the assembly program into your **8051 microcontroller**.
2. **Connect the LED:**
 - Connect the anode (positive side) of an LED to **P1.0** and the cathode (negative side) to **ground**.
3. **Run the Program:**

- Once the program is uploaded to the microcontroller and the power is applied, the LED connected to **P1.0** will blink at the interval defined by the timer.

Important Notes:

- The delay depends on the **Timer's initial value** and the **clock speed** of the microcontroller.
- You can use different combinations of **TH0** and **TL0** values to adjust the delay. Smaller values will create shorter delays, and larger values will create longer delays.
- This code can be modified to use **Timer 1** instead of **Timer 0** by changing the **TMOD** register appropriately (MOV TMOD, #0x10 for Timer 1, Mode 1).

Conclusion:

These programs demonstrate how to use the **8051 Timer** in **Mode 1** (16-bit mode) to generate delays by toggling an LED connected to **P1.0**. You can experiment with different initial values for **TH0** and **TL0** to modify the delay time and make the LED blink faster or slower.

10. Write a program to configure and use the serial port of the 8051 microcontrollers for serial communication.

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

To configure and use the **serial port** of the **8051 microcontroller** for serial communication (e.g., transmitting data via UART), we need to configure the **serial control register (SCON)** and set up the **baud rate** using **Timer 1**. Below is a simple **serial communication** program using the **8051 microcontroller** to transmit data via the serial port.

In this example, we will use the **8051 UART** for transmitting a byte of data (0x41 = 'A') and receiving a byte.

Serial Communication Program for 8051

Code:

assembly

Copy

ORG 0H ; Program starts at address 0

START:

MOV SCON, #50H ; Configure Serial Control register:

; 0101 0000 - Mode 1 (8-bit UART)

; 0101 0000 - 8 data bits, 1 stop bit, REN enabled (Receiver enabled)

MOV TMOD, #20H ; Timer 1 in Mode 2 (8-bit auto-reload)

MOV TH1, #0xFD ; Set Timer 1 reload value for 9600 baud rate (assuming 11.0592 MHz crystal)

MOV TL1, #0xFD ; Set Timer 1 initial value

SETB TR1 ; Start Timer 1 for baud rate generation

SETB TI ; Set Transmit Interrupt flag (indicating ready to transmit)

SETB REN ; Enable receiver (REN - Receiver Enable)

WAIT:

JNB TI, WAIT ; Wait for the transmit interrupt flag (TI) to be set (i.e., ready to transmit)

MOV SBUF, #'A' ; Send character 'A' (0x41)

CLR TI ; Clear Transmit Interrupt flag

; Wait for reception of a byte (optional)

JNB RI, WAIT_RECV ; Wait until the receive interrupt flag (RI) is set

WAIT_RECV:

MOV A, SBUF ; Read the received byte from the serial buffer into accumulator

MOV P1, A ; Display received byte on Port 1 (LED/Output)

SJMP WAIT ; Infinite loop to repeat the process

END

Explanation of the Program:

1. SCON (Serial Control Register):

- MOV SCON, #50H: Configure the serial port control register SCON.
 - **Bits 7-6:** Serial mode (Mode 1, 8 data bits, 1 stop bit).

- **Bit 5: REN** (Receiver Enable) – Allows the microcontroller to receive data.
 - **Bits 4-0:** Not used for Mode 1 in this case.
2. **Timer Configuration (for baud rate generation):**
 - MOV TMOD, #20H: Configure **Timer 1** in **Mode 2** (8-bit auto-reload mode).
 - MOV TH1, #0xFD and MOV TL1, #0xFD: Load the reload value into Timer 1 to generate the desired baud rate. Here, the baud rate is set to **9600** bps assuming a **11.0592 MHz** crystal oscillator (you may need to adjust this value for different crystal frequencies).
 - SETB TR1: Start Timer 1.
 - **Timer 1** generates the baud rate used for serial communication.
 3. **Transmission:**
 - SETB TI: Set the **TI (Transmit Interrupt flag)** to indicate that the serial port is ready for transmission.
 - MOV SBUF, #'A': Send the ASCII character 'A' (0x41) through the serial port.
 - CLR TI: Clear the **TI flag** after the byte is transmitted.
 4. **Reception:**
 - SETB REN: Enable the receiver to allow data reception.
 - JNB RI, WAIT_RECV: Wait until the **RI (Receive Interrupt flag)** is set, indicating that a byte has been received.
 - MOV A, SBUF: Once data is received, read it from the serial buffer (SBUF) into the accumulator A.
 - MOV P1, A: Display the received byte on **Port 1**. You can connect LEDs to **P1.0, P1.1, ..., P1.7** to observe the received data (or use a serial monitor on the PC if appropriate).
 5. **Infinite Loop:**
 - SJMP WAIT: The program loops continuously to wait for new data to be transmitted and received.

Key Registers for Serial Communication:

- **SCON (Serial Control Register):** Controls the configuration of the serial port.
 - **Bits 7-6:** Mode selection (Mode 1: 8 data bits, 1 stop bit).
 - **Bit 5: REN** (Receiver Enable).
 - **Bit 4: TI** (Transmit Interrupt flag).
 - **Bit 3: RI** (Receive Interrupt flag).
 - **Bit 0-1:** Reserved for serial mode.
- **SBUF (Serial Buffer Register):**
 - **SBUF** is used to transmit and receive data.
 - Write to **SBUF** to transmit data.
 - Read from **SBUF** to receive data.

Baud Rate Calculation:

For **9600 baud** with an **11.0592 MHz crystal oscillator**, the Timer 1 reload value is set to 0xFD. You can adjust this for different baud rates or crystal frequencies. The formula for the baud rate is:

$$\text{Baud Rate} = \frac{\text{Timer Frequency}}{32 \times (\text{Reload Value} + 1)}$$

Testing the Program:

1. **Hardware Setup:**
 - Connect the **TXD (transmit)** and **RXD (receive)** pins of the **8051** to a **serial communication interface**, such as a **USB-to-serial adapter** or a **serial monitor** on your PC.
 - You can use a **hyperterminal** or **serial monitor** to observe the transmitted data from the 8051 microcontroller.
2. **Program Execution:**
 - Once the program is loaded into the 8051 microcontroller, the character 'A' (0x41) will be transmitted via the serial port.
 - The program will wait for any received data and display it on **Port 1** (connected to LEDs or any other output).

Conclusion: This program demonstrates how to use the **8051 serial port** for basic **serial communication**. It transmits a byte (0x41, 'A') and waits to receive a byte, displaying the received data

on **Port 1**. You can adjust the baud rate and modify the program for various serial communication tasks, such as sending and receiving strings or handling more complex protocols.

11. Write simple programs to demonstrate data transfer instructions using the 8051- instruction set

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

mov , xch ,swap , push , pop

```
ORG 0000h
mov a,#05
mov b,#03
mov 10h,a
mov r1,10h
End
```

```
ORG 0000h
Mov a,#06h
Mov b,#07h
Xch a, r2
End
```

```
ORG 0000h
Mov a,#08
Mov r1,#04h
Swap a,r1
End
```

```
ORG 0000h
Mov a,#08
Mov r1,#04h
Push a
End
```

```
ORG 0000h
Mov a,#08
Mov r1,#04h
Pop a
End
```

Result :

12. Write simple programs to demonstrate arithmetic, and logical instructions using the 8051-instruction set

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

Arithmetic instruction : add, addc , sub , mul , div , inc, dec

```
ORG 0000h
mov a,#05
mov b,#03
add a,b
mov 10h,a
mov r1,10h
end
```

```
ORG 0000h
mov a,#05
mov b,#03
addc a,b
mov 10h,a
mov r1,10h
end
```

```
ORG 0000h
mov a,#05
mov b,#03
subb a,b
mov 10h,a
mov r1,10h
end
```

```
ORG 0000h
mov a,#03
mov b,#03
mul a,b
mov p2,a
mov r3,p2
end
```

```
ORG 0000h
mov a,#08h
mov b,#02h
div a,b
mov 30h,a
mov p0,a
End
```

```
ORG 0000h
mov a,#08h
mov b,#02h
Inc a
Inc b
mov 30h,a
mov p0,a
Inc p0
Inc 30h
End
```

```
ORG 0000h
mov a,#08h
mov b,#02h
dec a
dec b
mov 30h,a
mov p0,a
dec p0
dec 30h
End
```

Logical instruction : anl , orl , xrl , cpl , clr , rl , rr , rlc , rrc

```
ORG 0000h
mov a,#01h
mov b,#01h
anl a,b
mov 40h,a
mov p3,a
end
```

```
ORG 0000h
mov a,#01h
mov b,#00h
orl a,b
mov 20h,a
mov p2,a
end
```

```
ORG 0000h
mov a,#01h
mov b,#01h
xrl a,b
mov 10h,a
mov p0,a
end
```

```
ORG 0000h
mov a,#01h
cpl a
mov p3,a
end
```

```
ORG 0000h
mov a,#10h
clr a
mov p1,a
end
```

```
ORG 0000h
mov a,#02h
rr a
mov 05h,a
end
```

```
ORG 0000h
mov a,#03h
rl a
mov 03h,a
end
```

```
ORG 0000h
mov a,#02h
rrc a
mov 05h,a
end
```

```

ORG 0000h
mov a,#03h
rlc a
mov 03h,a
end

```

Result :

13. Write a program to turn an LED on and off using a port pin of the 8051 microcontrollers.

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

Here's a simple program to turn an LED on and off using a port pin of the 8051 microcontroller. In this example, we will control an LED connected to P1.0 (the first pin of Port 1).

Program: Turn LED on and off using 8051 microcontroller

The LED will turn on for a certain period, then turn off for another period. This cycle will repeat indefinitely.

Code:

assembly

Copy

```
ORG 0H          ; Start of program memory
```

START:

```
    MOV P1, #00H    ; Set all pins of Port 1 to LOW (LED OFF)
```

```
    ACALL DELAY      ; Call delay function
```

```
    MOV P1, #01H    ; Set P1.0 to HIGH (LED ON)
```

```
    ACALL DELAY      ; Call delay function
```

```
    SJMP START       ; Infinite loop to repeat the process
```

; Simple delay subroutine

DELAY:

```
    MOV R2, #20      ; Load R2 with 20 (Outer loop count)
```

DELAY_LOOP_1:

```
    MOV R1, #255     ; Load R1 with 255 (Inner loop count)
```

DELAY_LOOP_2:

```
    NOP              ; No operation (just waste time)
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

```
    NOP
```

14. Write a program to read input from one port and send output to another port using the 8051 microcontrollers.

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

Port 0 input , Port 2 output

```
mov a,p0
```

```
mov p2,a
```

end

Result :

1. p0 =
2. Accumulator =
3. p2 =

Port 1 input , Port 3 output

mov a,p1

mov p3,a

end

Result :

1. p1 =
2. Accumulator =
3. p3 =

15. Write a program to blink an LED at regular intervals using the timer functionality of the 8051 microcontrollers.

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

To blink an **LED** at regular intervals using the **timer** functionality of the **8051 microcontroller**, we will use **Timer 0** or **Timer 1** to generate a delay, which controls how fast the LED blinks. In this program, we will use **Timer 0** in **Mode 1** (16-bit timer mode) to generate a delay, and toggle the LED at **regular intervals**.

Program to Blink an LED using Timer in 8051

In this program:

1. We will use **Timer 0** to create a delay.
2. The LED will be connected to **P1.0** (pin 0 of Port 1).
3. The LED will toggle between **ON** and **OFF** at regular intervals.

Code:

assembly

Copy

ORG 0H ; Start of program memory

START:

```
MOV P1, #00H ; Set Port 1 to LOW (turn off LED initially)
MOV TMOD, #01H ; Configure Timer 0 in Mode 1 (16-bit timer mode)
MOV TH0, #0xFC ; Load Timer 0 high byte (for a 1 second delay at 11.0592 MHz clock)
MOV TL0, #0x18 ; Load Timer 0 low byte
SETB TR0 ; Start Timer 0
```

BLINK:

```
JNB TF0, BLINK ; Wait for Timer 0 overflow (TF0 set)
CLR TF0 ; Clear the Timer 0 overflow flag
CPL P1.0 ; Toggle LED connected to P1.0 (ON/OFF)
MOV TH0, #0xFC ; Reload Timer 0 high byte (for next cycle)
MOV TL0, #0x18 ; Reload Timer 0 low byte (for next cycle)
SETB TR0 ; Restart Timer 0
SJMP BLINK ; Infinite loop to keep blinking the LED
```

END

Explanation of the Program:

1. Timer Initialization:

- MOV TMOD, #01H: This configures **Timer 0** in **Mode 1** (16-bit mode). In this mode, Timer 0 is a 16-bit timer that can count from 0 to 65535.
- MOV TH0, #0xFC and MOV TL0, #0x18: These load the **high byte (TH0)** and **low byte (TL0)** of **Timer 0** to generate a specific time delay. For this case, we are using a 1-second delay at a clock frequency of **11.0592 MHz**.
 - The values 0xFC and 0x18 create a 1-second delay when Timer 0 overflows.
- SETB TR0: Starts Timer 0 to begin counting.

2. LED Blinking Logic:

- JNB TF0, BLINK: This instruction checks the **TF0** flag (Timer 0 overflow flag). It waits until **TF0** is set, indicating that Timer 0 has overflowed and the delay has elapsed.
- CPL P1.0: This toggles the **P1.0** pin. **CPL** is the complement instruction, so it changes the state of the LED: if the LED is OFF, it turns ON, and if it is ON, it turns OFF.
- After the toggle, we **reload Timer 0** with the same values for **TH0** and **TL0** to repeat the delay cycle.
- SETB TR0: Starts Timer 0 again to continue the delay.

3. Infinite Loop:

- SJMP BLINK: This instruction makes the program loop back to the **BLINK** label, so the LED will continue to blink indefinitely.

4. LED Behavior:

- The LED will blink **ON** and **OFF** at regular intervals, each lasting approximately **1 second**, based on the delay generated by Timer 0.

Key Registers:

- **TMOD (Timer Mode Register)**: Configures the mode of the timers.
 - **Bits 0-3**: Timer 0 mode.
 - **Bits 4-7**: Timer 1 mode (not used in this program).
- **TH0 (Timer 0 High Byte) and TL0 (Timer 0 Low Byte)**: Set the initial values for Timer 0.
- **TF0 (Timer 0 Overflow Flag)**: A flag that is set when Timer 0 overflows.
- **TR0 (Timer 0 Run Control Bit)**: Starts and stops Timer 0.

Testing the Program:

1. Hardware Setup:

- Connect an **LED** (with a current-limiting resistor) to **P1.0** of the **8051 microcontroller**.
- If you are using a **real board** or **development environment**:
 - Use an **LED** and a **resistor** (typically 220Ω) to avoid damaging the LED.
 - The anode (positive side) of the LED is connected to **P1.0**, and the cathode (negative side) to **ground**.

2. Program Execution:

- When the program runs on the 8051, the **LED** will blink **ON** and **OFF** at **1-second intervals** based on the timer delay.

3. Adjusting the Delay:

- To change the blink speed, modify the values loaded into **TH0** and **TL0**. For example:
 - Decreasing the value of **TH0** and **TL0** will result in a faster blink.
 - Increasing these values will slow down the blink.

Conclusion: This program demonstrates how to use the **8051 timer** to generate regular time delays and toggle an **LED** at regular intervals. By configuring **Timer 0** in **Mode 1**, we can create a simple, efficient way to blink an LED using the microcontroller's internal timer.

16. Write small assembly programs using different types of instructions.

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

Arithmetic instruction : add, addc , subb ,mul , div

```
ORG 0000h
mov a,#03h
mov b,#04h
add a,b
mov 11h,a
mov r6,11h
end
```

```
ORG 0000h
mov a,#02h
mov b,#05h
addc a,b
mov 12h,a
mov r5,12h
end
```

```
ORG 0000h
mov a,#05h
mov b,#02h
subb a,b
mov 45h,a
mov p1,a
end
```

```
ORG 0000h
mov a,#05h
mov b,#02h
mul ab
mov 55h,a
mov r4,55h
end
```

```
ORG 0000h
mov a,#05h
mov b,#02h
div a,b
mov 70h,a
mov r0,70h
end
```

Logical instruction : anl , orl , xrl , cpl , rl , rr

```
ORG 0000h
mov a,#01h
mov b,#01h
anl a,b
mov 80h,a
mov p6,a
end
```

```
ORG 0000h
mov a,#01h
mov b,#00h
orl a,b
mov 90h,a
mov r7,90h
end
```

```
ORG 0000h
mov a,#02h
cpl a
mov r4,a
mov p5,r4
end
```

```

ORG 0000h
mov a,#03h
rr a
mov r3,a
mov p2,r3
end

```

```

ORG 0000h
mov a,#03h
rl a
mov r0,a
mov 15h,r0
end

```

Result :

17. Write a program to send data from the 8051 to a PC via the serial port.

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

To send data from the **8051 microcontroller** to a **PC** via the **serial port**, we need to configure the **serial communication** on the **8051**. This is done using the **Serial Control Register (SCON)** and the **Timer** for generating a baud rate. We'll use the **8051's UART** to send data byte by byte to the PC through the **TXD (Transmit Data)** pin.

Program to Send Data from 8051 to PC via Serial Port

In this program, we will:

- Configure the **serial port**.
- Set the baud rate.
- Transmit data (a string or a byte) from the **8051** to the **PC**.

Key Registers:

- SCON (Serial Control Register)**: This controls the serial communication settings.
- TMOD (Timer Mode Register)**: This configures the timers for generating the baud rate.
- TH1 and TL1 (Timer 1 High and Low Bytes)**: These set the baud rate.
- TI (Transmit Interrupt Flag)**: This flag indicates that the **TXD** pin is ready to send data.

Program Code:

assembly

Copy

```
ORG 0H      ; Start of program memory
```

START:

```
    ; Initialize Serial Port
```

```
    MOV SCON, #50H    ; Configure serial control: 8-bit data, 1 stop bit, 1 start bit, mode 1
```

```
    MOV TMOD, #20H    ; Timer 1 in mode 2 (auto-reload mode)
```

```
    MOV TH1, #0xFD    ; Set Timer 1 to generate 9600 baud rate (for 11.0592 MHz clock)
```

```
    MOV TL1, #0xFD    ; Reload Timer 1 low byte
```

```
    SETB TR1          ; Start Timer 1 (enables the baud rate generation)
```

```
    ; Wait for the UART to be ready
```

WAIT_FOR_READY:

```
    JNB TI, WAIT_FOR_READY ; Wait until the transmit interrupt flag (TI) is set (ready to send data)
```

```
    CLR TI             ; Clear the transmit interrupt flag
```

```
    ; Send a character (data) to the PC
```

```
    MOV SBUF, #'A'    ; Send the character 'A' (ASCII value 65)
```

```
    ; Wait for the UART to finish transmitting
```

WAIT_FOR_TRANSMIT:

```
    JNB TI, WAIT_FOR_TRANSMIT ; Wait for the transmission to complete (TI flag set)
```

```

CLR TI          ; Clear the TI flag
; Optionally, send more data or repeat the process
; For example, sending the next character:
MOV SBUF, #'B'  ; Send the character 'B' (ASCII value 66)
; Wait again for transmission to complete
WAIT_FOR_TRANSMIT_2:
JNB TI, WAIT_FOR_TRANSMIT_2
CLR TI
SJMP START      ; Repeat sending data (loop) or add more data transmission
END

```

Explanation of the Code:

1. Serial Port Initialization (SCON Register):

- MOV SCON, #50H: Configures the **serial control register (SCON)**.
 - **Bits 7-6:** 00 — Serial mode 1 (8-bit data, 1 stop bit, 1 start bit).
 - **Bit 5:** 0 — No parity.
 - **Bit 4:** 1 — Enable serial port.
 - **Bits 3-0:** 0000 — No interrupt enable.

2. Timer Initialization (TMOD, TH1, TL1):

- MOV TMOD, #20H: Configures **Timer 1** in **Mode 2** (auto-reload mode) to generate the baud rate.
- MOV TH1, #0xFD and MOV TL1, #0xFD: Loads **Timer 1** with values that produce a **9600 baud rate** (assuming a 11.0592 MHz clock).
 - These values are derived from the formula: Baud rate = Timer overflow frequency / (32 * (256 - TH1)), where TH1 and TL1 control the timer reload.
- SETB TR1: Starts Timer 1 to begin generating the baud rate.

3. Sending Data:

- MOV SBUF, #'A': Sends the ASCII character 'A' (or any other data) to the **serial buffer (SBUF)**. The data is then transmitted to the **TXD pin**.
- **Wait for Transmission:**
 - JNB TI, WAIT_FOR_READY: The program waits for the **TI (Transmit Interrupt flag)** to be set, which indicates that the **TXD pin** is ready to transmit another byte of data.
 - CLR TI: Clears the **TI** flag to indicate that the byte has been transmitted.

4. Repeat Data Transmission:

- After sending one character (e.g., 'A'), the program sends the next character (e.g., 'B') in the same way.
- The SJMP START instruction causes the program to loop and repeatedly send data.

How to Test the Program:

1. Hardware Setup:

- **8051 Microcontroller** is connected to a **PC** through the **serial port (RS232)**. You need to use a **level converter** if the 8051 does not have a direct **RS232** interface (e.g., using an **MAX232 IC**).
- **TXD (Transmit Data)** pin of the 8051 is connected to the **RXD (Receive Data)** pin of the PC.
- You will need to use a **terminal software** on the PC (like **HyperTerminal**, **PuTTY**, or **Tera Term**) to view the data being sent from the 8051.

2. Running the Program:

- When the program is executed, the 8051 will send the characters 'A' and 'B' to the PC.
- You should see the characters displayed on the terminal software on the PC.

3. Adjusting the Baud Rate:

- You can modify the values of **TH1** and **TL1** to change the baud rate. For instance:
 - For a **9600 baud rate** with a **11.0592 MHz** clock, **TH1 = 0xFD** and **TL1 = 0xFD**.
 - For a **19200 baud rate**, you would change the values of **TH1** and **TL1**.

4. Sending More Data:

- You can modify the code to send more data by adding more MOV SBUF, data instructions with different data values.

Conclusion:

This program demonstrates how to send data from the **8051 microcontroller** to a **PC** using the **serial communication** protocol. By configuring the **serial port** and using **Timer 1** to generate a baud rate, we can transmit data byte by byte. You can extend this program to send strings or multiple characters, making it useful for serial communication projects.

18. Write a program to display numbers 0-9 on a 7-segment display.

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

To display numbers **0-9** on a **7-segment display** using an **8051 microcontroller**, we need to map the numbers to their corresponding 7-segment display encoding. A **7-segment display** has 7 LEDs (labeled a to g) that can be turned on or off to represent different numbers.

The segments of a 7-segment display are typically controlled by turning on or off the appropriate segments. For common **anode 7-segment displays**, a segment is lit when its corresponding bit is **LOW**, and for **common cathode 7-segment displays**, a segment is lit when the corresponding bit is **HIGH**.

7-Segment Display Encoding for Numbers 0-9:

Below is the **binary encoding** for numbers 0-9 on a **7-segment display**:

Number Segments (abcdefg)

0	0x3F
1	0x06
2	0xB6
3	0x76
4	0x3E
5	0x7A
6	0x7F
7	0x06
8	0x7F
9	0x7E

Each hexadecimal value represents the binary state of the segments. For example, for number **0** (0x3F), the binary equivalent is 00111111, meaning that segments **a**, **b**, **c**, **d**, **e**, and **f** are on, and segment **g** is off.

Program to Display Numbers 0-9 on a 7-Segment Display using 8051:

Assumptions:

- The **7-segment display** is connected to **Port 1** (P1.0 to P1.6).
- The display is a **common cathode** 7-segment display, which means we will write **HIGH** to light up a segment.
- We will cycle through the numbers **0-9** with a small delay to display each number.

Program Code:

assembly

Copy

ORG 0H ; Start of program memory

START:

MOV P1, #00H ; Set all Port 1 pins LOW (turn off all segments)

; Display numbers 0 to 9

MOV DPTR, #NUMBERS ; Load the address of the numbers' lookup table

DISPLAY_LOOP:

MOV A, @DPTR ; Fetch the next number from the lookup table

MOV P1, A ; Display the number on Port 1 (7-segment display)

ACALL DELAY ; Call the delay subroutine

INC DPTR ; Move to the next number in the lookup table

MOV A, DPL ; Check if we have reached the end of the table

ORL A, DPH

JNZ DISPLAY_LOOP ; If not end of table, keep displaying numbers

```

    SJMP START      ; Repeat the loop (from the start)
; 7-segment display encoding for numbers 0-9
NUMBERS:
    DB 0x3F        ; 0
    DB 0x06        ; 1
    DB 0xB6        ; 2
    DB 0x76        ; 3
    DB 0x3E        ; 4
    DB 0x7A        ; 5
    DB 0x7F        ; 6
    DB 0x06        ; 7
    DB 0x7F        ; 8
    DB 0x7E        ; 9
; Delay subroutine for visible display
DELAY:
    MOV R2, #255    ; Outer loop
DELAY_LOOP1:
    MOV R1, #255    ; Inner loop
DELAY_LOOP2:
    NOP             ; No Operation (used for delay)
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    DJNZ R1, DELAY_LOOP2 ; Inner loop countdown
    DJNZ R2, DELAY_LOOP1 ; Outer loop countdown
    RET
END

```

Explanation of the Code:

1. **Port Initialization (MOV P1, #00H):**
 - Initially, we set **Port 1** to **LOW** (MOV P1, #00H), ensuring that all the segments of the 7-segment display are turned off.
2. **Lookup Table (NUMBERS):**
 - The NUMBERS table contains the **7-segment encoding** for the numbers **0 to 9**. Each byte represents the pattern of segments to be lit up on the 7-segment display.
 - For example, 0x3F corresponds to the binary value 00111111, which lights up segments **a, b, c, d, e, f** for displaying **0**.
3. **Displaying Numbers (MOV A, @DPTR and MOV P1, A):**
 - DPTR points to the NUMBERS table. We fetch the value from the table using MOV A, @DPTR and load it into **A**.

- The value in **A** is then transferred to **P1** to display the corresponding number on the 7-segment display.
- 4. **Delay (ACALL DELAY):**
 - After displaying each number, a **delay** is introduced using the DELAY subroutine. This ensures the number stays visible for a short time before displaying the next number.
- 5. **Looping through Numbers (INC DPTR and MOV A, DPL):**
 - After each display, we increment the **data pointer (DPTR)** to point to the next number in the table. The loop continues until all numbers from **0 to 9** are displayed.
 - If the end of the table is reached (i.e., all 10 numbers have been displayed), the program restarts from the beginning with SJMP START.
- 6. **Delay Subroutine (DELAY):**
 - The delay is created using nested loops. Each NOP instruction adds a small delay, and the loops repeat until the desired delay time has passed.

Testing the Program:

1. **Hardware Setup:**
 - **7-segment display** is connected to **Port 1** of the 8051.
 - Each segment (a, b, c, d, e, f, g) is connected to one pin of **Port 1**.
 - For a **common cathode** display, the pins are **active HIGH**, meaning the segments light up when the corresponding bit is **1**.
2. **Run the Program:**
 - When the program runs, the numbers **0-9** will display sequentially on the 7-segment display. Each number will stay visible for a brief period before transitioning to the next.
3. **Adjusting the Delay:**
 - You can modify the **delay loop** to increase or decrease the time each number stays visible on the display.

Conclusion:

This program demonstrates how to display numbers **0-9** on a **7-segment display** using the **8051 microcontroller**. The numbers are displayed by writing the corresponding 7-segment encoding to **Port 1**, and a simple delay ensures that each number stays on the display for a short period before moving to the next.

19. Implement a program to create a simple counter using the 7-segment display.

- a. Write algorithm of program .
- b. Draw flowchart of program .
- c. Write Assembly Language Program .

To implement a **simple counter** using a **7-segment display** with the **8051 microcontroller**, we will display numbers from **0 to 9** in a continuous loop, incrementing the counter each time. The program will use a **7-segment display** connected to the **8051 microcontroller** to show the count.

Steps:

1. Use a **7-segment display** to display numbers from **0 to 9**.
2. Increment the counter every time a number is displayed.
3. Use a simple delay to make the count visible to the human eye before it changes.
4. Once the counter reaches **9**, it will reset back to **0**.

Program to Implement a Simple Counter using 7-Segment Display on 8051

Assumptions:

- A **common cathode** 7-segment display is connected to **Port 1** of the **8051 microcontroller**.
- The **counter** will be displayed from **0 to 9**.
- We will use a simple **delay** to ensure that the numbers are displayed for a visible period.

7-Segment Encoding for Numbers 0-9:

Number 7-Segment Encoding (Hex)

0	0x3F
1	0x06
2	0xB6
3	0x76
4	0x3E

Number 7-Segment Encoding (Hex)

5	0x7A
6	0x7F
7	0x06
8	0x7F
9	0x7E

Each hexadecimal value corresponds to the state of the segments **a to g**. For example:

- **0x3F**: This will light up the segments **a, b, c, d, e, f** to display **0**.

Program Code:

assembly

Copy

ORG 0H ; Start of program memory

START:

MOV P1, #00H ; Set all Port 1 pins LOW (turn off all segments initially)

MOV DPTR, #NUMBERS ; Load the address of the numbers' lookup table

COUNTER_LOOP:

MOV A, @DPTR ; Fetch the next number from the lookup table

MOV P1, A ; Display the number on Port 1 (7-segment display)

ACALL DELAY ; Call the delay subroutine

INC DPTR ; Move to the next number in the lookup table

MOV A, DPL ; Check if we have reached the end of the table

ORL A, DPH

JNZ COUNTER_LOOP ; If not end of table, keep displaying numbers

; Reset the counter and restart

MOV DPTR, #NUMBERS ; Reload the address of the numbers' lookup table

SJMP COUNTER_LOOP ; Repeat the counter loop

; 7-segment display encoding for numbers 0-9

NUMBERS:

DB 0x3F ; 0

DB 0x06 ; 1

DB 0xB6 ; 2

DB 0x76 ; 3

DB 0x3E ; 4

DB 0x7A ; 5

DB 0x7F ; 6

DB 0x06 ; 7

DB 0x7F ; 8

DB 0x7E ; 9

; Delay subroutine for visible display

DELAY:

MOV R2, #255 ; Outer loop

DELAY_LOOP1:

MOV R1, #255 ; Inner loop

DELAY_LOOP2:

NOP ; No Operation (used for delay)

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

NOP

```

NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
DJNZ R1, DELAY_LOOP2 ; Inner loop countdown
DJNZ R2, DELAY_LOOP1 ; Outer loop countdown
RET
END

```

Explanation of the Code:

1. **Port Initialization (MOV P1, #00H):**
 - This initializes **Port 1** to **LOW** at the beginning of the program, ensuring that all the segments of the 7-segment display are turned off initially.
2. **Lookup Table (NUMBERS):**
 - The **NUMBERS** table stores the **7-segment display encoding** for numbers **0 to 9**. Each value represents the state of the segments **a to g**.
 - For example, 0x3F corresponds to the number **0**, lighting up segments **a, b, c, d, e, f**.
3. **Displaying Numbers (MOV A, @DPTR and MOV P1, A):**
 - The **DPTR** register points to the **NUMBERS** table. We fetch the value of the current number using **MOV A, @DPTR** and load it into register **A**.
 - We then transfer the value from **A** to **P1**, which displays the corresponding number on the 7-segment display.
4. **Delay (ACALL DELAY):**
 - After each number is displayed, a **delay** is introduced to make the number visible for a brief period. This delay is handled by the **DELAY** subroutine, which uses nested loops to create a simple time delay.
5. **Looping (INC DPTR and MOV A, DPL):**
 - After displaying a number, we increment the **data pointer (DPTR)** to point to the next number in the **NUMBERS** table. This continues until the counter reaches the end of the table (after displaying **9**).
6. **Resetting the Counter:**
 - Once the counter reaches **9**, we reload the **DPTR** register with the address of the **NUMBERS** table (**MOV DPTR, #NUMBERS**), and the counter starts again from **0**.
7. **Delay Subroutine (DELAY):**
 - The **DELAY** subroutine creates a time delay using nested loops. Each **NOP** instruction takes a small amount of time, and the loops repeat for the desired delay.

Testing the Program:

1. **Hardware Setup:**
 - **7-segment display** is connected to **Port 1** of the 8051 microcontroller.
 - Each segment of the display (labeled **a to g**) is connected to the pins **P1.0 to P1.6**.
 - For a **common cathode** display, each segment is turned **on** by writing **HIGH** to the corresponding pin.
2. **Running the Program:**
 - When the program is executed, the counter will display the numbers **0 to 9** sequentially on the 7-segment display. Each number will stay visible for a short period before the next number is displayed.
 - After displaying **9**, the counter will reset to **0** and continue.
3. **Adjusting the Delay:**
 - The delay duration can be adjusted by modifying the inner loop of the **DELAY** subroutine. A larger value for the loop variables will make the delay longer, causing the numbers to stay on the display for a longer time.

Conclusion:

This program demonstrates how to implement a simple counter that displays numbers **0 to 9** on a **7-segment display** using the **8051 microcontroller**. The program uses a **lookup table** to map the numbers to the corresponding 7-segment display patterns and a simple **delay** to make each number visible before incrementing the counter.

20. Write a program to display characters and strings on an LCD. Create a program to scroll text on the LCD.

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

To display characters and strings on an LCD and implement text scrolling using an **8051 microcontroller**, we need to understand the basic working of the LCD and the commands required to interface with it.

LCD Overview:

- An **LCD** (Liquid Crystal Display) typically has a **16x2** matrix, meaning it can display 16 characters in each of the 2 rows. Each character on the LCD is represented by a **5x7 dot matrix**.
- We use a set of commands to control the LCD:
 - Command Mode**: Used to send control instructions (e.g., clear screen, set cursor position).
 - Data Mode**: Used to send character data for display.

Pin Configuration:

- RS (Register Select)**: Determines whether data or command is being sent.
 - 0**: Command register.
 - 1**: Data register (character or string).
- RW (Read/Write)**: Specifies whether we are reading from or writing to the LCD.
 - 0**: Write.
 - 1**: Read.
- E (Enable)**: Latches data to the LCD when a rising edge is applied.
- D0-D7**: Data pins for the LCD.

For simplicity, we assume a **4-bit mode** for communication, meaning only the higher nibble (4 bits) of the data is sent at a time.

LCD Initialization:

- First, we initialize the LCD in 4-bit mode.
- Then, we send commands to set the display to 2 lines, 5x7 characters, and turn on the display with no cursor.

Program Description:

- Displaying a string**: We'll send each character from the string to the LCD in data mode.
- Scrolling the text**: We will continuously shift the text one position to the left on the LCD.

Program Code for Displaying and Scrolling Text on LCD:

Assumptions:

- The LCD is connected to **Port 1** (P1.0 to P1.7).
- RS** is connected to **P3.0**, **RW** to **P3.1**, and **E** to **P3.2**.

Program Code:

assembly

Copy

ORG 0H ; Start of program memory

START:

MOV P1, #00H ; Set Port 1 to low initially (LCD reset)

MOV P3, #00H ; Set control pins (RS, RW, E) to low

; Initialize the LCD

ACALL LCD_INIT

; Display a string on the LCD

MOV DPTR, #STRING

ACALL DISPLAY_STRING

```

    ; Scroll the string on the LCD
    MOV R0, #00
SCROLL_LOOP:
    MOV DPTR, #STRING
    ACALL SCROLL_TEXT
    NOP
    SJMP SCROLL_LOOP
; Subroutine to initialize the LCD
LCD_INIT:
    ; Set 4-bit mode, 2 lines, 5x7 font
    ACALL LCD_COMMAND
    MOV A, #0x02
    ACALL SEND_CMD
    MOV A, #0x28 ; 4-bit mode, 2-line display, 5x7 font
    ACALL SEND_CMD
    MOV A, #0x0C ; Display on, cursor off
    ACALL SEND_CMD
    MOV A, #0x06 ; Increment cursor after each character
    ACALL SEND_CMD
    MOV A, #0x01 ; Clear display
    ACALL SEND_CMD
    RET
; Subroutine to send a command to the LCD
SEND_CMD:
    CLR P3.0 ; RS = 0 (Command mode)
    CLR P3.1 ; RW = 0 (Write mode)
    SETB P3.2 ; E = 1 (Enable)
    MOV P1, A ; Send command to Port 1
    CLR P3.2 ; E = 0 (Enable)
    NOP
    NOP
    RET
; Subroutine to send data (character) to the LCD
SEND_DATA:
    SETB P3.0 ; RS = 1 (Data mode)
    CLR P3.1 ; RW = 0 (Write mode)
    SETB P3.2 ; E = 1 (Enable)
    MOV P1, A ; Send data to Port 1
    CLR P3.2 ; E = 0 (Enable)
    NOP
    NOP
    RET
; Subroutine to display a string on the LCD
DISPLAY_STRING:
    MOV R0, #00 ; Initialize index to 0
DISPLAY_LOOP:
    MOV A, @DPTR ; Fetch character from string
    INC DPTR ; Move to next character
    ORL A, A ; Check if it's the null character (end of string)
    JZ DISPLAY_END ; If null, end of string
    ACALL SEND_DATA
    SJMP DISPLAY_LOOP
DISPLAY_END:
    RET
; Subroutine to scroll the text
SCROLL_TEXT:

```

```

MOV R0, #16 ; Start from first position (16th character)
MOV R1, #00 ; Set initial offset for scrolling
SCROLL_LOOP:
MOV A, @DPTR ; Fetch character from string
INC DPTR ; Move to next character
ORL A, A ; Check if it's the null character (end of string)
JZ SCROLL_END ; If null, end of scroll
ACALL SEND_DATA
INC R1
DJNZ R1, SCROLL_LOOP
SCROLL_END:
RET
; String to be displayed
STRING:
DB 'Hello World!' ; Example text to be displayed on the LCD
END

```

Explanation of the Code:

1. Initialization (LCD_INIT):

- The LCD is initialized in 4-bit mode with the following settings:
 - **0x02**: Initialize in 4-bit mode.
 - **0x28**: Set the LCD to 2 lines and 5x7 characters.
 - **0x0C**: Turn on the display and turn off the cursor.
 - **0x06**: Set the cursor to increment after each character.
 - **0x01**: Clear the display.
- These commands are sent to the LCD using the **SEND_CMD** subroutine.

2. Displaying a String (DISPLAY_STRING):

- The program reads each character from the **STRING** array using the **DPTR** register and sends it to the LCD using the **SEND_DATA** subroutine.
- The string is displayed until a null byte (0x00) is encountered, which indicates the end of the string.

3. Scrolling Text (SCROLL_TEXT):

- The **SCROLL_TEXT** subroutine shifts the text one position to the left by sending the text to the LCD with an incrementing offset. The text scrolls from the starting position and continues until the entire string is moved.
- The loop handles the shifting by sending each character one by one, and after the last character, it loops back.

4. Control Pins:

- The **RS**, **RW**, and **E** pins control whether data or commands are sent to the LCD and whether the operation is read or written.
- **RS** determines if we are sending a command or data (for characters).
- **RW** is set to **0** for write operations.
- **E** is toggled to latch the data or command.

Testing the Program:

1. Hardware Setup:

- **RS** is connected to **P3.0**, **RW** to **P3.1**, and **E** to **P3.2** of the **8051**.
- **Data Pins** (D0-D7) are connected to **Port 1** (P1.0 to P1.7).
- **LCD** is powered and properly connected to the microcontroller.

2. Running the Program:

- The program initializes the LCD and displays the string "Hello World!" on the screen.
- After displaying the string, the program will scroll the text from right to left, showing each character for a brief period.

Conclusion:

This program demonstrates how to display and scroll text on an **LCD** using the **8051 microcontroller**. It initializes the LCD in 4-bit mode, displays a string, and scrolls it across the screen. The text scrolls by continuously shifting the characters from right to left.

21. Write a program to read temperature data from the sensor. Display the temperature on an LCD.

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

To read temperature data from a sensor and display it on an LCD using an **8051 microcontroller**, we will use a **temperature sensor**. One of the most common temperature sensors is the **LM35** (analog temperature sensor). The **LM35** provides an output voltage proportional to the temperature in degrees Celsius (1 mV per °C).

Hardware Setup:

- LM35 Temperature Sensor:** This sensor outputs an analog voltage corresponding to the temperature.
- ADC:** Since the **8051** microcontroller does not have a built-in **Analog-to-Digital Converter (ADC)**, we need to use an external ADC (e.g., **ADC0808**) to convert the analog voltage from the **LM35** into a digital value that can be read by the **8051**.
- LCD Display:** We'll use a 16x2 LCD to display the temperature in Celsius.

Working Flow:

- The **LM35** sensor outputs an analog voltage, which is converted to a digital value using the external **ADC**.
- The **8051** reads the digital value from the **ADC** and processes it to get the temperature in Celsius.
- The temperature is displayed on the **16x2 LCD**.

Steps to Achieve This:

- Read the ADC value:** The **ADC0808** or any similar ADC will convert the voltage from the **LM35** to a digital value.
- Process the ADC data:** Convert the ADC value into temperature in Celsius (since the LM35 provides 1mV/°C).
- Display the value on the LCD:** Send the calculated temperature to the LCD.

Pin Configuration for LCD:

- RS, RW, and E** control pins for the LCD will be connected to **P3.0, P3.1, and P3.2** of the **8051** microcontroller.
- Data pins (D0-D7)** will be connected to **Port 1 (P1.0 to P1.7)** of the **8051**.

Program Code for Reading Temperature and Displaying on LCD:

Assumptions:

- LM35** is connected to the ADC input.
- ADC0808** is used to convert the analog signal to a digital signal. The **ADC0808** has an 8-bit output, meaning the temperature will be represented by an 8-bit value.
- The **temperature** is displayed on a **16x2 LCD** connected to **Port 1** of the **8051**.
- The **ADC0808** is interfaced with **8051** using control pins connected to **Port 2**.

assembly

Copy

ORG 0H ; Start of program memory

START:

MOV P1, #00H ; Initialize Port 1 for LCD (set to LOW)

MOV P3, #00H ; Initialize Port 3 for control pins (RS, RW, E)

; Initialize the LCD

ACALL LCD_INIT

; Initialize ADC0808 (assuming appropriate control pins are configured)

; Select ADC channel, start conversion and read the result

; Assume ADC conversion is complete and result is available in a register (e.g., R0)

; Start ADC conversion process (for simplicity, this is a mock step)

ACALL READ_ADC

; Convert ADC result to temperature (LM35 -> 1 mV per °C)

MOV A, R0 ; Move ADC value (0-255) into Accumulator (A)

MOV B, #0x32 ; Assume R0 value is in the range 0 to 255 (ADC value)

MUL AB ; Multiply the ADC value by 0.5 (just for simplicity here, actual conversion depends on the ADC)

```

; Display temperature value on the LCD
ACALL DISPLAY_TEMP
; Infinite loop to keep displaying the temperature
SJMP START
; Subroutine to initialize the LCD
LCD_INIT:
    ACALL LCD_COMMAND
    MOV A, #0x02 ; Initialize in 4-bit mode
    ACALL SEND_CMD
    MOV A, #0x28 ; 2-line, 5x7 font
    ACALL SEND_CMD
    MOV A, #0x0C ; Display on, cursor off
    ACALL SEND_CMD
    MOV A, #0x06 ; Auto-increment cursor
    ACALL SEND_CMD
    MOV A, #0x01 ; Clear display
    ACALL SEND_CMD
    RET
; Subroutine to send command to LCD
SEND_CMD:
    CLR P3.0 ; RS = 0 (Command mode)
    CLR P3.1 ; RW = 0 (Write mode)
    SETB P3.2 ; E = 1 (Enable)
    MOV P1, A ; Send command to Port 1
    CLR P3.2 ; E = 0 (Disable)
    NOP
    NOP
    RET
; Subroutine to send data (character) to LCD
SEND_DATA:
    SETB P3.0 ; RS = 1 (Data mode)
    CLR P3.1 ; RW = 0 (Write mode)
    SETB P3.2 ; E = 1 (Enable)
    MOV P1, A ; Send data to Port 1
    CLR P3.2 ; E = 0 (Disable)
    NOP
    NOP
    RET
; Subroutine to read ADC value (simulated here)
READ_ADC:
    MOV R0, #0x32 ; Mock ADC value (simulating the reading from ADC0808)
    RET
; Subroutine to display temperature on LCD
DISPLAY_TEMP:
    MOV A, R0 ; Get the temperature (from ADC value or calculation)
    ACALL SEND_DATA ; Display the temperature on the LCD
    RET
END

```

Explanation of the Program:

1. Initialization:

- The **LCD** is initialized in 4-bit mode. The display is configured with **2 lines** and **5x7 font**.
- **Port 1** and **Port 3** are configured for **data and control** pins to communicate with the LCD.

2. ADC Read:

- The program assumes that the **ADC0808** is used to convert the analog signal from the **LM35** into a digital value.

- The **READ_ADC** subroutine simulates the process of reading the ADC value. In practice, you would interact with the ADC by sending control signals to the **ADC0808** (e.g., select channel, start conversion) and then reading the output from the ADC.

3. **Temperature Calculation:**

- The temperature is calculated by processing the ADC value. If the **LM35** sensor outputs **1mV per °C**, you would need to convert the ADC value (which represents the voltage) into a temperature value.
- Here, for simplicity, we assume that the conversion process is done correctly and directly use the ADC value for temperature. In a real scenario, you would scale the ADC value based on the resolution of the ADC.

4. **Displaying the Temperature:**

- The **DISPLAY_TEMP** subroutine sends the temperature to the LCD, converting it to ASCII characters.

Testing the Program:

1. **Hardware Setup:**

- The **LM35** is connected to the ADC input.
- The **ADC0808** is used for conversion. Its data output is connected to **Port 1** of the **8051**.
- **Control pins (CS, WR, RD)** of the **ADC0808** are connected to **Port 2** of the **8051**.
- The **LCD** is connected to **Port 1** of the **8051** as described earlier.

2. **Running the Program:**

- The program initializes the LCD and starts reading the temperature data from the **LM35** via the **ADC**.
- The calculated temperature is displayed on the LCD.

3. **Calibration:**

- The ADC value can be calibrated by testing the actual output of the **LM35** at known temperatures and adjusting the program to provide accurate readings.

Conclusion:

This program demonstrates how to read temperature data from an **LM35** sensor, convert the analog data to a digital format using an **ADC0808**, and display the temperature on an **LCD** using the **8051 microcontroller**.

22. **Write a program to rotate the stepper motor in different directions. Implement speed control for the stepper motor.**

- Write algorithm of program .
- Draw flowchart of program .
- Write Assembly Language Program .

To control a **stepper motor** in different directions and implement **speed control** using the **8051 microcontroller**, we will need to understand the working of a **stepper motor** and how it is controlled through different pins.

Stepper Motor Control:

A stepper motor moves in discrete steps, and each step corresponds to a small movement of the motor shaft. The most common type of stepper motor is the **4-phase** motor, which requires **4 control signals** to rotate the motor in steps.

For this example, we will use the **unipolar stepper motor** and drive it using the **4 control pins**. The basic control of a stepper motor is achieved by energizing the coils in a specific sequence.

Pin Configuration:

We will assume the following configuration:

- The **stepper motor** is connected to **Port 1 (P1.0 to P1.3)** for controlling the four coils.
- The **speed control** is implemented using a **delay** between the steps.

Speed Control:

Speed control can be achieved by adjusting the delay between each step of the motor. A smaller delay results in faster rotation, while a larger delay results in slower rotation.

Rotation Directions:

- **Clockwise (CW):** A specific sequence of energizing the coils will rotate the motor in a clockwise direction.
- **Counterclockwise (CCW):** Reversing the coil energizing sequence will rotate the motor in the opposite direction.

Program Code for Controlling the Stepper Motor:

This program will rotate the motor clockwise and counterclockwise, and the speed of rotation will be controlled by adjusting the delay between steps.

Program Code:

assembly

Copy

ORG 0H ; Start of program memory

START:

MOV P1, #00H ; Initialize Port 1 for stepper motor (set to low initially)

MOV P3, #00H ; Initialize Port 3 for control pins

; Rotate stepper motor clockwise

MOV R0, #0x03 ; Set a small delay (lower value = faster speed)

ACALL ROTATE_CW

; Rotate stepper motor counterclockwise

MOV R0, #0x03 ; Set a small delay for speed control

ACALL ROTATE_CCW

SJMP START ; Infinite loop to keep rotating

; Subroutine to rotate the stepper motor clockwise

ROTATE_CW:

MOV R1, #4 ; 4 steps for a full rotation

ROTATE_CW_LOOP:

MOV A, R1

DEC A

MOV R1, A

; Rotate step 1

MOV P1, #0x01 ; Coil 1 ON

ACALL DELAY

; Rotate step 2

MOV P1, #0x03 ; Coil 1 & 2 ON

ACALL DELAY

; Rotate step 3

MOV P1, #0x02 ; Coil 2 ON

ACALL DELAY

; Rotate step 4

MOV P1, #0x06 ; Coil 2 & 3 ON

ACALL DELAY

; Check if we need to continue rotation

JNZ ROTATE_CW_LOOP

RET

; Subroutine to rotate the stepper motor counterclockwise

ROTATE_CCW:

MOV R1, #4 ; 4 steps for a full rotation

ROTATE_CCW_LOOP:

MOV A, R1

DEC A

MOV R1, A

; Rotate step 1

MOV P1, #0x06 ; Coil 2 & 3 ON

ACALL DELAY

; Rotate step 2

MOV P1, #0x02 ; Coil 2 ON

ACALL DELAY

; Rotate step 3

MOV P1, #0x03 ; Coil 1 & 2 ON

ACALL DELAY

; Rotate step 4

MOV P1, #0x01 ; Coil 1 ON

ACALL DELAY

; Check if we need to continue rotation

JNZ ROTATE_CCW_LOOP

RET

; Subroutine to introduce delay for speed control

DELAY:

MOV R2, R0 ; Load the delay value

DELAY_LOOP:

MOV R3, #0xFF ; Inner loop to create a small delay

DELAY_INNER_LOOP:

NOP ; No Operation - just wasting time

NOP

NOP

NOP

DJNZ R3, DELAY_INNER_LOOP

DJNZ R2, DELAY_LOOP

RET

END

Explanation of the Code:

1. Port Initialization:

- **Port 1 (P1.0 to P1.3)** is initialized to control the stepper motor coils. Initially, the pins are set to low to turn off the coils.

2. Rotating the Motor Clockwise (ROTATE_CW):

- The subroutine **ROTATE_CW** implements the clockwise rotation sequence for the stepper motor. The coils are energized in a specific sequence:
 - **Step 1:** Activate coil 1.
 - **Step 2:** Activate coil 1 and coil 2.
 - **Step 3:** Activate coil 2.
 - **Step 4:** Activate coil 2 and coil 3.
- After each step, a delay is introduced using the **DELAY** subroutine.

3. Rotating the Motor Counterclockwise (ROTATE_CCW):

- The subroutine **ROTATE_CCW** implements the counterclockwise rotation sequence. The coils are energized in the reverse order of the clockwise sequence:
 - **Step 1:** Activate coil 2 and coil 3.
 - **Step 2:** Activate coil 2.
 - **Step 3:** Activate coil 1 and coil 2.
 - **Step 4:** Activate coil 1.

4. Speed Control (DELAY):

- The **DELAY** subroutine is used to control the speed of the motor by introducing a delay between each step. The delay is determined by the value in **R0**, which can be adjusted to change the motor speed.

- The larger the value in **R0**, the slower the motor will rotate, and vice versa.

5. Main Loop:

- The main loop rotates the stepper motor in both clockwise and counterclockwise directions by calling the **ROTATE_CW** and **ROTATE_CCW** subroutines. After one complete cycle, it repeats indefinitely.

Testing the Program:

1. Hardware Setup:

- **Step 1:** Connect the stepper motor to **Port 1 (P1.0 to P1.3)**.
- **Step 2:** Ensure the **stepper motor** is properly powered (typically 5V for small motors).
- **Step 3:** Optionally, use **external transistors** or a **motor driver IC (e.g., L298N)** to drive the stepper motor if the current required by the motor exceeds what the **8051** can provide.

2. Adjusting Speed:

- The delay value in **R0** controls the speed. Try different values for **R0** (e.g., #0x02, #0x03, #0x05) to change the rotation speed.

3. Rotating the Motor:

- The motor will rotate clockwise and counterclockwise continuously.

Conclusion:

This program demonstrates how to control a **stepper motor** using the **8051 microcontroller**. It covers:

- **Clockwise and counterclockwise rotation** using a 4-phase sequence.
- **Speed control** by adjusting the delay between steps.

You can easily modify the delay values to adjust the speed or the number of steps for a complete rotation.